

PRACTICAL NO. 4

AIM

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

- Kill Processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

2. Write a program for process creation using C

- Orphan Process
- Zombie Process

3. Create the process using fork () system call.

- Child Process creation
- Parent Process creation
- PPID and PID

PERFORMANCE

1.

- Kill Processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

OUTPUT

```
309@309-BY-OEM:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1         0  0  16:19 ?        00:00:07 /sbin/init splash
root           2         0  0  16:19 ?        00:00:00 [kthreadd]
root           3         2  0  16:19 ?        00:00:00 [pool_workqueue_release]
root           4         2  0  16:19 ?        00:00:00 [kworker/R-rcu_gp]
root           5         2  0  16:19 ?        00:00:00 [kworker/R-sync_wq]
root           6         2  0  16:19 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root           7         2  0  16:19 ?        00:00:00 [kworker/R-slub_flushwq]
root           8         2  0  16:19 ?        00:00:00 [kworker/R-netns]
root          10         2  0  16:19 ?        00:00:00 [kworker/0:0H-events_highpri]
root          13         2  0  16:19 ?        00:00:00 [kworker/R-mm_percpu_wq]
root          14         2  0  16:19 ?        00:00:00 [rcu_tasks_kthread]
root          15         2  0  16:19 ?        00:00:00 [rcu_tasks_rude_kthread]
root          16         2  0  16:19 ?        00:00:00 [rcu_tasks_trace_kthread]
root          17         2  0  16:19 ?        00:00:00 [ksoftirqd/0]
root          18         2  0  16:19 ?        00:00:00 [rcu_preempt]
root          19         2  0  16:19 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
root          20         2  0  16:19 ?        00:00:00 [rcu_exp_gp_kthread_worker]
root          21         2  0  16:19 ?        00:00:00 [migration/0]
root          22         2  0  16:19 ?        00:00:00 [idle_inject/0]
root          23         2  0  16:19 ?        00:00:00 [cpuhp/0]
root          24         2  0  16:19 ?        00:00:00 [cpuhp/1]
root          25         2  0  16:19 ?        00:00:00 [idle_inject/1]
root          26         2  0  16:19 ?        00:00:00 [migration/1]
root          27         2  0  16:19 ?        00:00:00 [ksoftirqd/1]
root          29         2  0  16:19 ?        00:00:00 [kworker/1:0H-events_highpri]
root          30         2  0  16:19 ?        00:00:00 [cpuhp/2]
root          31         2  0  16:19 ?        00:00:00 [idle_inject/2]
root          32         2  0  16:19 ?        00:00:00 [migration/2]
```

```
m309      7713      7697  0 17:10 pts/0    00:00:00 grep --color=auto firefox
m309@m309-BY-OEM:~$ kill 7713
bash: kill: (7713) - No such process

m309@m309-BY-OEM:~$ pkill firefox
```

2. Write a program for process creation using C

- Orphan Process
- Zombie Process

OUTPUT

1. ORPHAN PROCESS

```
m309@m309-BY-OEM:~$ Child PID: 8566
New Parent PID: 2119

m309@m309-BY-OEM:~$ nano orphan.c
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    if (fork() == 0) {
        sleep(5);
        printf("Child PID: %d\n", getpid());
        printf("New Parent PID: %d\n", getppid());
    } else {
        printf("Parent exiting\n");
    }
    return 0;
}
```

```
New Parent PID: 2119

m309@m309-BY-OEM:~$ nano orphan.c
m309@m309-BY-OEM:~$ gcc orphan.c -o orphan
m309@m309-BY-OEM:~$ ./orphan
Parent exiting
m309@m309-BY-OEM:~$ Child PID: 9309
New Parent PID: 2119
```

2. ZOMBIE PROCESS

```
m309@m309-BY-OEM:~$ nano zombie.c
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child exiting\n");
        return 0;
    } else {
        // Parent process
        printf("Parent sleeping, child becomes zombie\n");
        sleep(30); // VERY IMPORTANT
    }
    return 0;
}
```

[Read 17 lines]

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set M

```
m309@m309-BY-OEM:~$ nano zombie.c
m309@m309-BY-OEM:~$ gcc zombie.c -o zombie
m309@m309-BY-OEM:~$ ./zombie
Parent sleeping, child becomes zombie
Child exiting
```

3. Create the process using fork ()system call.

- Child Process creation
- Parent Process creation
- PPID and PID

OUTPUT

```
m309@m309-BY-OEM:~$ nano fork_demo.c
m309@m309-BY-OEM:~$ gcc fork_demo.c -o fork_demo
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    } else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
    }
    return 0;
}
```

[Read 16 lines]

Help Write Out Where Is Cut Execute Location Undo Set M

```
m309@m309-BY-OEM:~$ gcc fork_demo.c -o fork_demo
m309@m309-BY-OEM:~$ ./fork_demo
Parent exiting
m309@m309-BY-OEM:~$ Child PID: 8566
New Parent PID: 2119
```

4.INFINITE LOOP PROCESS

OUTPUT

